PID CONTROL IN ROBOTICS

IME 307 - Computer Programming

PROJECT ABSTRACT & THEORY

1. Introduction to PID Control in Robotics

In robotic control systems, precise trajectory tracking is a fundamental requirement. Whether controlling a robotic arm manipulator or an autonomous vehicle, motors must drive components to a specific Setpoint (SP) smoothly and accurately.

A Proportional-Integral-Derivative (PID) controller is a continuous feedback mechanism that solves this problem by minimizing the tracked system Error (e(t)), which is the difference between the target position and the actual position.

The Mathematical Formula

The control output signal u(t) sent to the robot's actuators is calculated as:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt}$$

Where u(t) is the control output, e(t) is the current error

The Three Components

1. Proportional Gain (Kp):

Corrects the system based on the current magnitude of the error. High Kp values ensure a fast response but can cause severe system oscillations.

2. Integral Gain (Ki):

Corrects based on accumulated past errors, eliminating steady-state offset to ensure the robot lands precisely on target.

3. Derivative Gain (Kd):

Predicts future error by measuring the system's current rate of change. It acts as an algorithmic brake, preventing overshoot and mechanical structural fatigue.

SECTION 2: STANDALONE C++ SOURCE CODE

Save the C++ code as:

pid_robot.cpp

[Complete C++ implementation of PID Feedback Controller for Robot Arm Tracking]

SECTION 3: COMPILATION & USER GUIDE

Follow these step-by-step instructions to compile and execute the PID controller simulation. This guide is designed for beginners with minimal command-line experience.

STEP 1: PREPARE YOUR COMPUTER

Requirement: Install MSYS2 Compiler

Before you can compile C++ code, you need a C++ compiler. MSYS2 is a free compiler for Windows.

✓ Download MSYS2 from: https://www.msys2.org/

✓ Install it (use default settings)

✓ After installation, launch MSYS2 UCRT64 (you'll see a black terminal window)

STEP 2: OPEN TERMINAL & CREATE PROJECT FOLDER

Open your MSYS2 UCRT64 terminal window. You should see a prompt that looks like this:

RLS@DESKTOP-GIGOUDF UCRT64 /home/user$

Now create a project folder by typing this command:
$ mkdir /c/cpp_project

Then navigate into that folder:

$ cd /c/cpp_project

You should now see the prompt has changed to show you're in the cpp_project folder.

STEP 3: PLACE YOUR CODE FILE

Open Notepad or VS Code on your computer and create a new file. Copy the C++ code (pid_robot.cpp) into this file.

Save the file with these exact settings:

✓ File name:
pid_robot.cpp
✓ Save location:
C:\cpp_project

STEP 4: COMPILE THE PROGRAM

In your terminal, you should already be in the /c/cpp_project folder. Now compile the code by typing this command:

$ g++ pid_robot.cpp -o pid_robot.exe

Press Enter and wait. The compiler will translate your C++ code into an executable program.

If successful, you'll see the $ prompt return. If there are errors, check your code for typos.

STEP 5: RUN THE PROGRAM

Now execute your compiled program by typing:

$ ./pid_robot.exe

Press Enter. Your PID controller simulation will start running!

USING THE PID CONTROLLER PROGRAM

Understanding the Menu

When you run the program, you'll see a menu with options. The program shows:

DEPARTMENT OF ENGINEERING -- ROBOTICS SOFTWARE CORNERSelect an Engineering Module to Solve:1. Robotics & Controls: PID Controller Simulation2. Electrical: RLC Circuit AC Impedance Calculator3. Fluid Mechanics: Pipe Friction Loss4. Exit Program

How to Use: PID Controller (Option 1)

1. Type '1' and press Enter
The program will ask for three inputs:

✓ Enter target value (Setpoint): 100
→ This is where you want the robot arm to reach (in cm)
✓ Enter current starting value (Process Variable): 0
→ This is where the robot starts (usually 0)
✓ Enter simulation steps: 10
→ How many iterations the simulation should run

Understanding the Output Table

After you enter your values, the program displays a table with these columns:

Column	Meaning	Formula	Example	Trend	Goal

Step	Iteration number	1,2,3...	Step 1	Increases	N/A

Error	Gap to target	Target - Current	100 cm	Decreases	→ 0
P-Term	Proportional control	Kp × Error	160	Decreases	Immediate response
I-Term	Accumulated error	Ki × Sum(Errors)	4	Increases	Eliminates offset
D-Term	Rate of change	Kd × dError/dt	120	Variable	Prevents overshoot
Motor Output	Total control signal	P+I+D	284	Decreases	Drives motor

KEY INSIGHT: Watch how the Error column gets smaller with each step. That's the PID controller working! It's bringing the robot closer to the target position.

How to Return to Menu

After the simulation completes, you'll be back at the main menu. You can try another module (1, 2, or 3) or type 4 to exit the program.

Troubleshooting

Problem: 'command not found' error

Solution: Make sure you're in the /c/cpp_project folder. Type: cd /c/cpp_project

Problem: 'No such file or directory'

Solution: Make sure pid_robot.cpp is saved in C:\cpp_project\ folder

Problem: Compilation errors

Solution: Check for typos in the code. Every semicolon and bracket must be correct.

The readme file below:

https://github.com/Powerful-2/odin-recipes/blob/main/README.md?plain=1

At its core, this project solves the problem of imprecise machine motion and collision risks in automation

In our world today, machines cannot think like humans; a motor only understands raw electricity. If you apply power blindly, a robotic arm will swing too fast, slam into its target, and break itself or hurt someone.

Here is exactly how the two systems fix this in the real world:

1. The Real-World Problem Solved by the PID Controller

Machines naturally suffer from overshoot and oscillation (shaking back and forth).

If an automated machine moves a heavy component, momentum makes it bypass the target.

- **The Solution:** Your PID controller calculates the exact braking force needed in real-time [1, 2]. It ensures that things like

automated surgery robots, factory packing arms, and elevator cabins stop exactly where they are supposed to, smoothly and without vibrating.

2. The Real-World Problem Solved by the Braking Simulator

Autonomous systems face the constant risk of unexpected obstacles. If a self-driving delivery vehicle or a factory warehouse robot encounters a human worker or a wall, it needs to know when and how hard to brake based on its speed.

- **The Solution:** Your obstacle avoidance code creates a dynamic safety envelope. It solves the mathematical problem of matching deceleration to distance, ensuring the vehicle stops with room to spare instead of skidding into a crash.

How It Changes the Narrative in Nigeria Right Now

Beyond global tech, your project solves local, structural economic challenges across Nigeria today:

- ***Saves Millions in Equipment Repairs:*** In industrial hubs like Ikeja, Ilupeju, and Agbara, manufacturing plants running automated bottling or packaging lines lose millions when machine parts collide due to poor calibration. Your software prevents that mechanical damage.
- ***Secures Unmanned Logistics:*** With the rise of drone logistics for medical deliveries in remote parts of Nigeria (like Zipline's operations), PID controllers keep delivery drones stable against unpredictable tropical wind gusts so they don't crash.
- ***Reduces Post-Harvest Losses in Agriculture:*** Automated sorting machines driven by PID loops allow commercial farms in states like Benue and Kaduna to rapidly sort grains and tubers by weight and size without bruising the produce, boosting food security.

CONCLUSION

The PID Feedback Controller is a powerful tool for robotic control systems. By following this guide, you can compile and execute the simulation, observe how proportional, integral, and derivative gains work together, and understand the mathematical principles behind precision robot arm tracking. Practice with different input values and gains to deepen your understanding of control systems engineering.

The project demonstrates that implementing automated C++ control loops, specifically PID controllers and obstacle avoidance systems, effectively bridges software algorithms with physical hardware to enhance safety and efficiency.

Recommendations for future development include Hardware In-The-Loop (HIL) deployment, integrating automated gain tuning, and employing multi-sensor systems to improve reliability in industrial environments.

--- END OF DOCUMENT ---